Bahria University
Department of Computer Science

Skill Barter Platform

Submitted to: Aima Zahoor
Semester: Spring 2026
Class: BS-CS 4-B
Deadline: May 10, 2026

Group Members

Sr.	Name	Enrollment No.
1	Muhammad Sadiq Aslam Khan	01-134242-091
2	Malik Junaid Ahmed	01-134242-061
3	Bilawal Khokhar	01-134242-036
4	Shah Murad	01-134242-113

May 2026

Contents

1 Problem Statement **3**

2 Project Scope **3**

3 Functional Requirements **3**

4 Non-Functional Requirements **4**

5 Use Case Diagram **5**

6 Use Case Description Tables **6**

7 Context Diagram (DFD Level 0) **11**

8 Data Flow Diagram – Level 1 **12**

9 Data Flow Diagram – Level 2 **13**

10 Class Diagram **14**

11 Sequence Diagram **15**

12 Activity Diagram **15**

13 State Machine Diagram **17**

14 Architecture **18**
14.1 Layer Breakdown 18

15 Architecture Diagram **19**

16 Request Flow **20**

17 Design Decisions **20**

18 Test Plan **21**
18.1 Unit Tests 21
18.2 Integration Tests 22
18.3 System Tests 22

19 Version Control **24**

20 Project Planning **25**
20.1 Gantt Chart 25
20.2 PERT Chart 25

21 Risk Analysis **26**

22 Team Roles **26**

Milestone 1: Requirements

Problem Statement

Students, freelancers, and working professionals often need new skills but cannot afford paid tutoring or courses. At the same time, plenty of people have skills they would teach but no way to find someone who wants to learn them.

A skill barter platform fixes this by letting users swap skills directly. Someone who knows web development and wants to learn graphic design finds another user with the opposite setup and arranges a trade. No money changes hands. The system handles matching, messaging, scheduling, and feedback so users do not have to sort it out themselves.

Every existing option is either a paid platform, a disorganised social media group, or a freelance marketplace built around money. Nothing is built specifically around skill exchange.

Project Scope

Users can register, build profiles, and list skills they offer and skills they want. The system matches users with complementary skill sets. Users send and receive barter requests, communicate through a messaging interface, and schedule sessions. After a session, both parties rate each other. Admins manage users, resolve disputes, and monitor platform activity.

The system does not handle payments, real-time video calls, or third-party login. Hardware integration is out of scope. The platform is documented as a web-based application.

Functional Requirements

FR ID	Requirement
FR-01	Users can register with name, email, and password
FR-02	Users can log in and log out securely
FR-03	Users can create and edit their profile including skills offered and skills wanted
FR-04	Users can search and browse other users by skill
FR-05	Users can send a barter request to another user
FR-06	Users can accept or reject incoming barter requests
FR-07	Matched users can communicate through an in-platform messaging system
FR-08	Users can schedule a session with date, time, and mode (online or in-person)
FR-09	Users can rate and review each other after a completed session
FR-10	Users can view their barter history

FR ID	Requirement
FR-11	Admin can view, suspend, or delete any user account
FR-12	Admin can view all active and completed barter sessions
FR-13	Admin can resolve reported disputes between users
FR-14	Users can report another user for misconduct
FR-15	System sends notifications for new requests, messages, and session reminders

Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Search results load within 2 seconds under normal load
NFR-02	Scalability	System supports up to 10,000 concurrent users
NFR-03	Security	Passwords are stored as hashed values, never plain text
NFR-04	Security	Role-based access control separates user and admin functions
NFR-05	Usability	New users can register and send a barter request within 5 minutes
NFR-06	Availability	System uptime is 99.5% or higher
NFR-07	Maintainability	System uses a layered architecture so modules can be updated independently
NFR-08	Reliability	All barter request state changes are logged and recoverable
NFR-09	Compatibility	Platform is accessible on modern web browsers without plugins
NFR-10	Privacy	User personal data is not visible to other users without consent

Use Case Diagram

Actors: Registered User, Admin, Notification Service (external)

Use Cases: Register/Login, Manage Profile, Search Users by Skill, Send Barter Request, Accept/Reject Request, Message Matched User, Schedule Session, Rate and Review, View Barter History, Report User, Manage Users (Admin), Resolve Dispute (Admin), View Platform Reports (Admin), Send Notifications (Notification Service)

Use Case Description Tables

UC-01: Register / Login

Field	Description
Use Case ID	UC-01
Actor	Registered User
Description	New user creates an account. Existing user logs in with credentials.
Preconditions	User has browser access to the platform
Normal Flow	1. User opens platform. 2. New user fills name, email, password and registers. 3. Existing user enters credentials and logs in. 4. System redirects to dashboard.
Alternative Flow	If email is already registered, system prompts login instead.
Exception Flow	After 3 failed login attempts, account is locked temporarily.
Postconditions	User is authenticated and can access platform features.

UC-02: Manage Profile

Field	Description
Use Case ID	UC-02
Actor	Registered User
Description	User creates or updates profile with bio, skills offered, and skills wanted.
Preconditions	User is logged in
Normal Flow	1. User goes to profile page. 2. Edits bio and skill lists. 3. Saves changes. 4. System updates profile.
Alternative Flow	Multiple skills can be added in both categories.
Exception Flow	Required fields left empty prompt user before saving.
Postconditions	Profile is updated and searchable by other users.

UC-03: Search Users by Skill

Field	Description
Use Case ID	UC-03
Actor	Registered User

Field	Description
Description	User searches for other users who offer a specific skill.
Preconditions	User is logged in and other users have skills listed
Normal Flow	1. User enters skill in search bar. 2. System returns matching users. 3. User views profiles.
Alternative Flow	Results can be filtered by rating or availability.
Exception Flow	No matching results shows a no results message.
Postconditions	User views profiles and can send a barter request.

UC-04: Send Barter Request

Field	Description
Use Case ID	UC-04
Actor	Registered User
Description	User sends a barter request proposing a skill exchange to another user.
Preconditions	User is logged in and has at least one skill listed
Normal Flow	1. User views another profile. 2. Clicks Send Barter Request. 3. Selects skill offered and skill wanted. 4. Adds optional message and submits. 5. Recipient is notified.
Alternative Flow	Request can be edited before submitting.
Exception Flow	No skill listed prompts user to update profile first.
Postconditions	Request created with pending status and recipient notified.

UC-05: Accept or Reject Request

Field	Description
Use Case ID	UC-05
Actor	Registered User
Description	User reviews an incoming barter request and accepts or rejects it.
Preconditions	User has at least one pending incoming request
Normal Flow	1. User opens request inbox. 2. Views request details. 3. Clicks Accept or Reject. 4. Sender is notified.
Alternative Flow	User can send a message before deciding.

Field	Description
Exception Flow	None.
Postconditions	If accepted, both users proceed to messaging and scheduling.

UC-06: Schedule Session

Field	Description
Use Case ID	UC-06
Actor	Registered User
Description	Matched users agree on a date, time, and mode for the skill exchange session.
Preconditions	Barter request has been accepted
Normal Flow	1. Either user opens the barter thread. 2. Proposes date, time, and mode. 3. Other user confirms. 4. System records session and sends reminders.
Alternative Flow	Users can reschedule before the session date.
Exception Flow	No confirmation within 7 days marks the session as expired.
Postconditions	Session is scheduled and both users receive reminders.

UC-07: Rate and Review

Field	Description
Use Case ID	UC-07
Actor	Registered User
Description	After a completed session, both users rate each other and leave an optional review.
Preconditions	Session has been marked as completed
Normal Flow	1. System prompts both users to rate. 2. User gives 1 to 5 stars and optional written review. 3. System saves and displays on profile.
Alternative Flow	User can skip written review and submit rating only.
Exception Flow	Rating window closes after 7 days if not submitted.
Postconditions	Rating recorded and profile score updated.

UC-08: Report User

Field	Description
Use Case ID	UC-08
Actor	Registered User
Description	User reports another user for misconduct such as no-show, harassment, or fake skills.
Preconditions	User is logged in
Normal Flow	1. User clicks Report on profile or barter thread. 2. Selects reason and adds optional details. 3. System logs report and sends to admin queue.
Alternative Flow	None.
Exception Flow	Duplicate report on same user is blocked by system.
Postconditions	Report logged and admin notified.

UC-09: Manage Users (Admin)

Field	Description
Use Case ID	UC-09
Actor	Admin
Description	Admin views all users and can suspend or delete accounts.
Preconditions	Admin is logged in
Normal Flow	1. Admin opens user management panel. 2. Searches or filters users. 3. Views profile and activity. 4. Suspends or deletes if needed. 5. User is notified.
Alternative Flow	Admin can reinstate a suspended account.
Exception Flow	Admin cannot delete their own account through this panel.
Postconditions	Account status updated across the platform.

UC-10: Resolve Dispute (Admin)

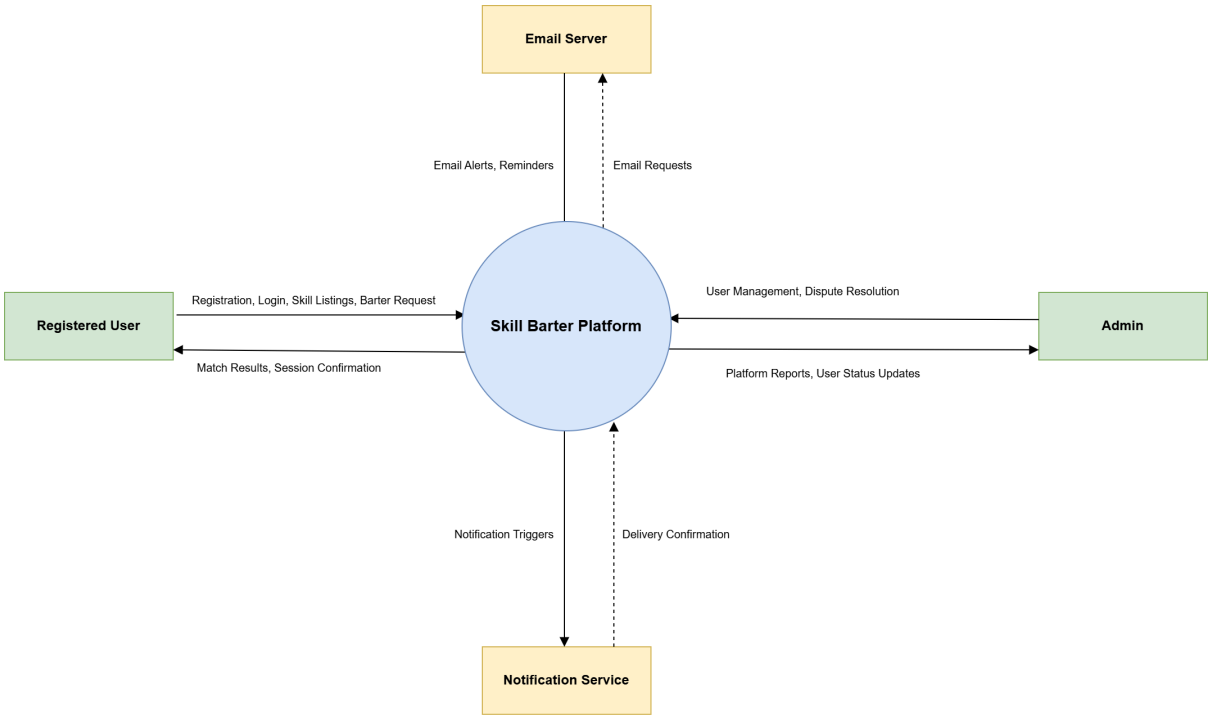
Field	Description
Use Case ID	UC-10
Actor	Admin
Description	Admin reviews a reported dispute and takes action.

Field	Description
Preconditions	A report is in the admin queue
Normal Flow	1. Admin opens dispute queue. 2. Reviews report and evidence. 3. Contacts parties if needed. 4. Closes with resolution: warning, suspension, or no action. 5. Both users notified.
Alternative Flow	Admin can escalate to a higher-level admin.
Exception Flow	None.
Postconditions	Dispute closed with recorded resolution.

Milestone 2: Modeling

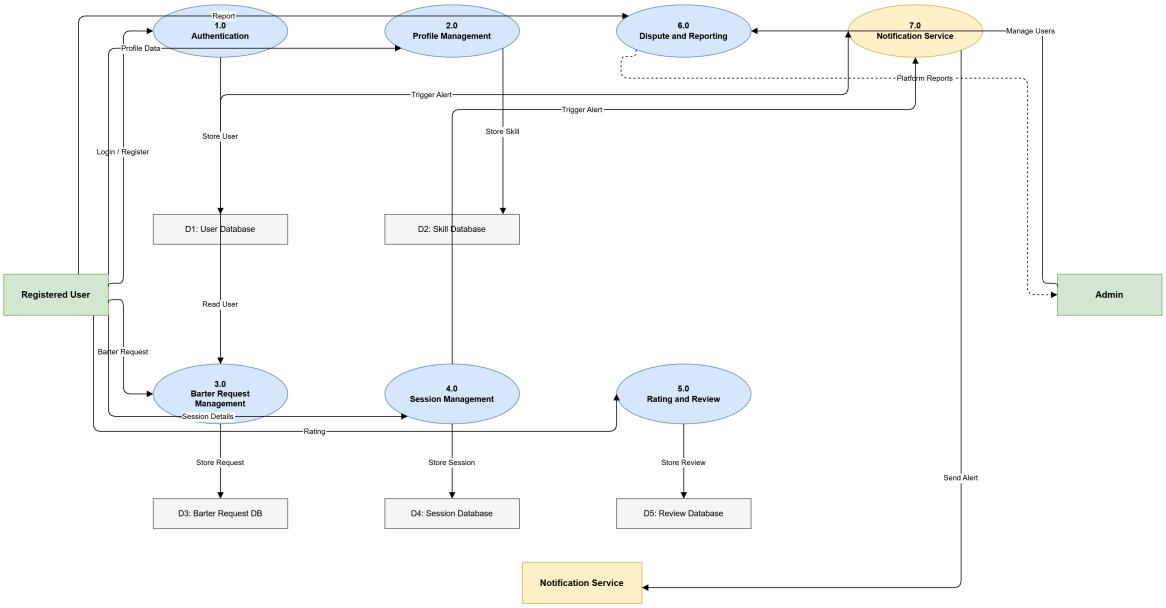
Context Diagram (DFD Level 0)

The context diagram is a Level 0 DFD. It treats the platform as a single black box and shows all external entities that interact with it. The Registered User sends registration data, skill listings, barter requests, session details, and ratings in. The system sends back match results, session confirmations, and notifications. The Admin pushes user management commands and dispute resolutions in and gets platform reports back. The Notification Service receives alert triggers and returns delivery confirmations. The Email Server handles outbound alerts and reminders.



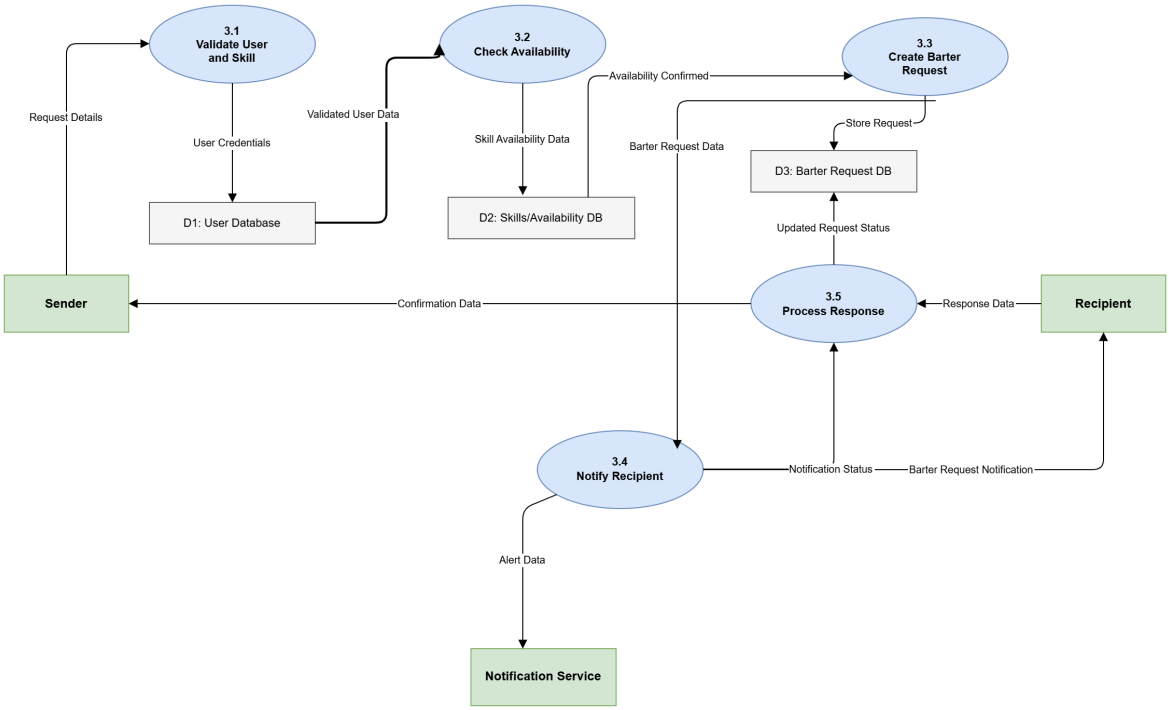
Data Flow Diagram – Level 1

The Level 1 DFD expands the system into seven internal processes. Authentication handles login and registration and writes to the User Database. Profile Management handles skill updates and writes to the Skill Database. Barter Request Management receives requests from users, reads the User Database, writes to the Barter Request Database, and triggers the Notification process. Session Management writes to the Session Database and also triggers notifications. Rating and Review writes to the Review Database. Dispute and Reporting receives reports from users and commands from Admin and returns platform reports back. The Notification process receives triggers and sends alerts to the external Notification Service.



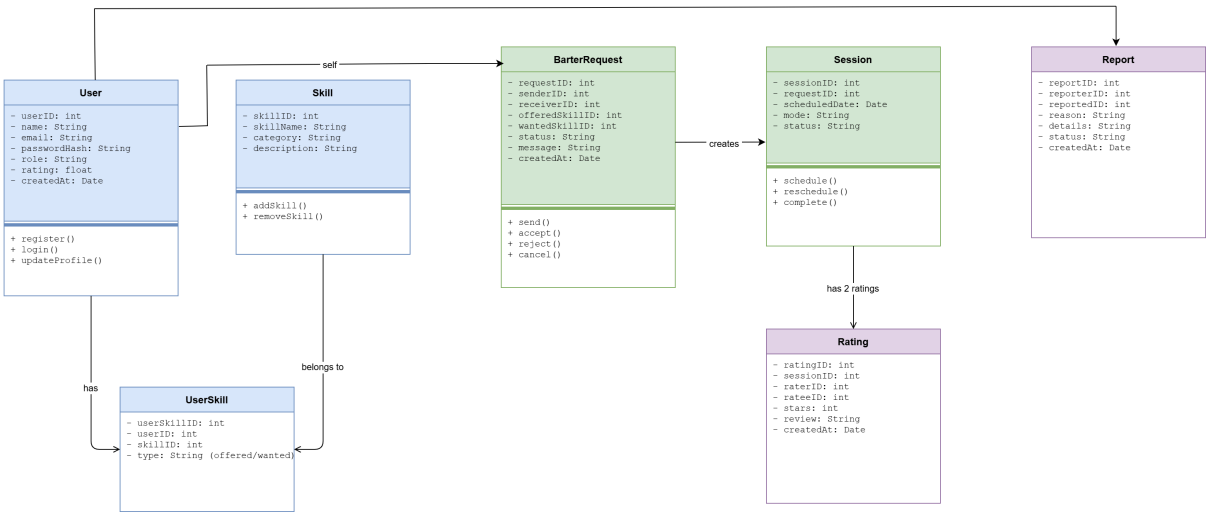
Data Flow Diagram – Level 2

The Level 2 DFD zooms into Process 3.0 Barter Request Management and breaks it into five sub-processes. Process 3.1 receives request details from the Sender, queries the User Database to verify credentials, and returns validated user data. Process 3.2 checks recipient skill availability against the Skills/Availability Database. Process 3.3 creates the barter request and writes it to the Barter Request Database. Process 3.4 notifies the Recipient and sends an alert to the Notification Service. Process 3.5 receives the Accept or Reject response from the Recipient, updates the Barter Request Database with the new status, and sends confirmation data back to the Sender.



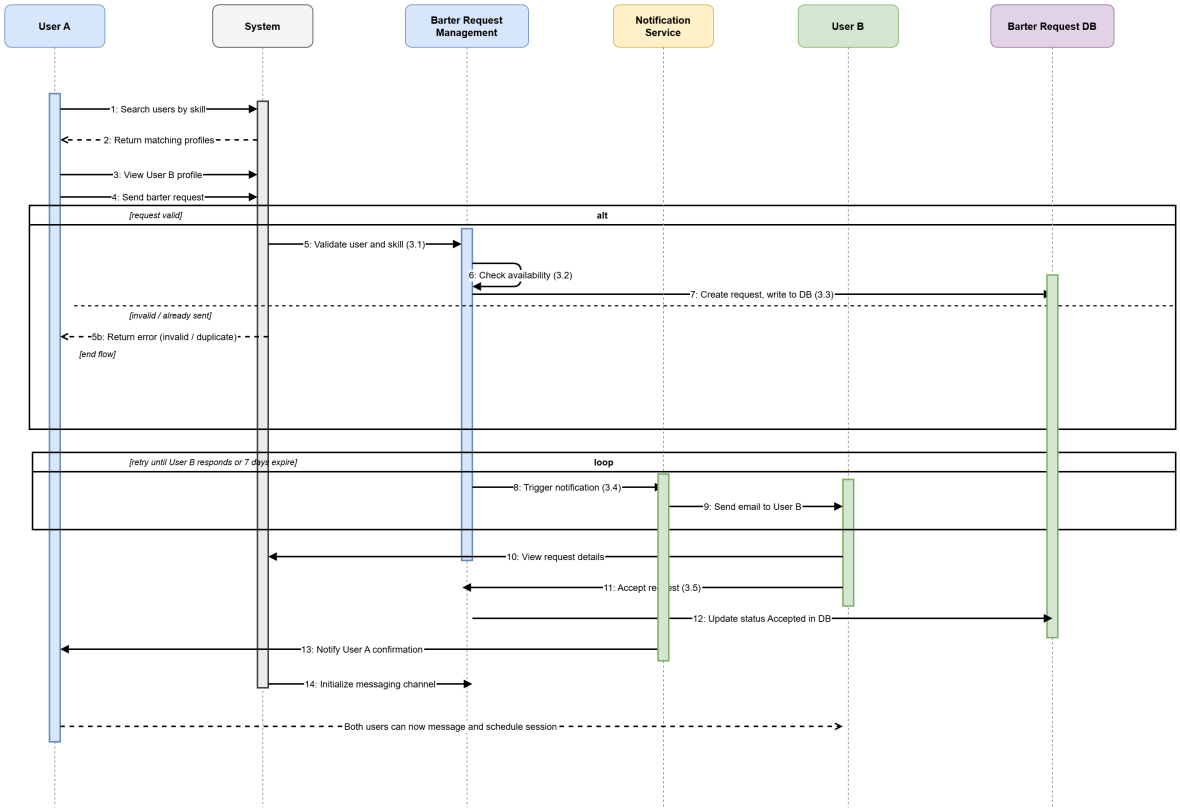
Class Diagram

The diagram covers seven classes. User holds identity and auth data with register, login, and updateProfile methods. Skill stores name, category, and description. UserSkill is an association class linking User and Skill with a type field marking the skill as offered or wanted. BarterRequest records sender, receiver, the two skills exchanged, status, and message with send, accept, reject, and cancel methods. Session is created on acceptance and holds date, mode, and status. Each Session produces two Rating records storing stars and an optional review. Report records misconduct filed by users and is managed by Admin.



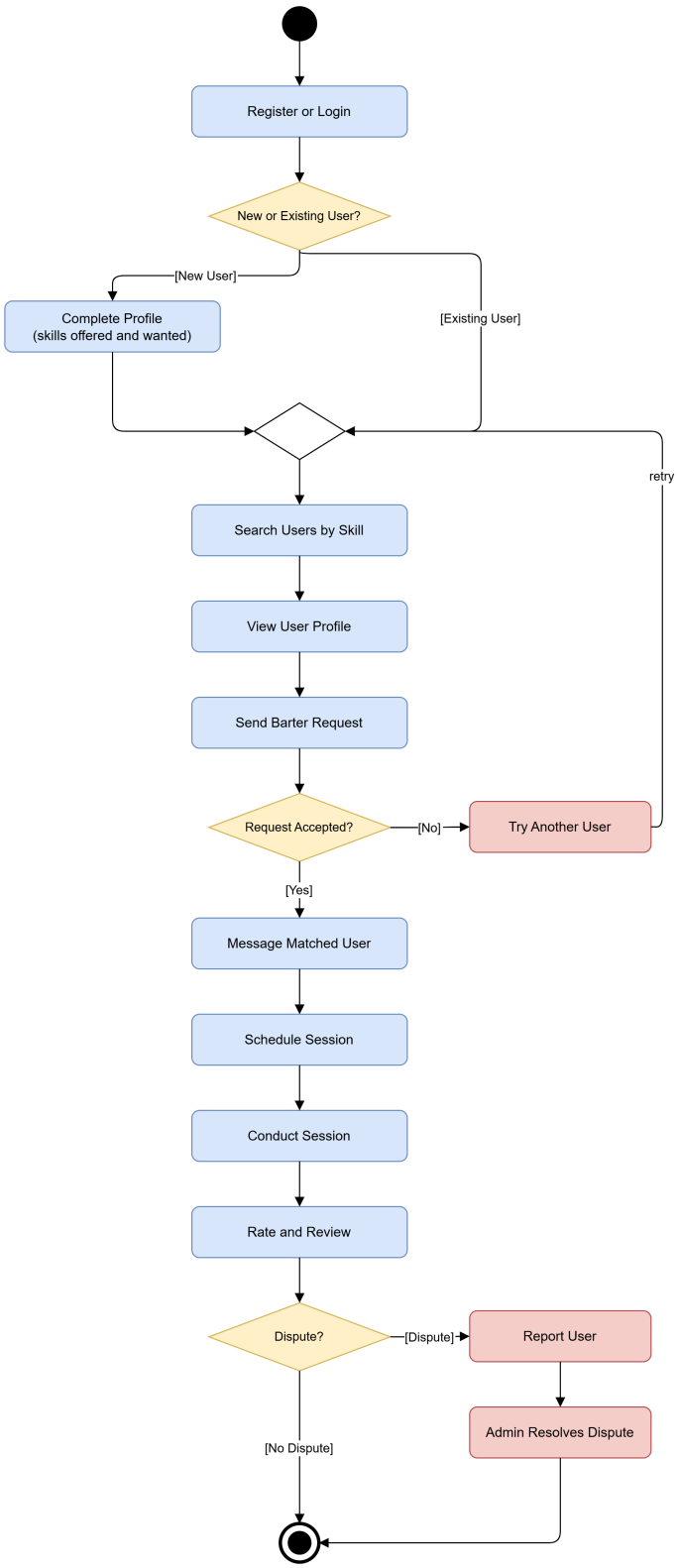
Sequence Diagram

The diagram traces the Send and Accept Barter Request flow across six lifelines: User A, System, Barter Request Management, Notification Service, User B, and Barter Request DB. User A searches and views User B's profile. After sending a request the system validates it through sub-processes 3.1 and 3.2. If valid, 3.3 creates the request and writes it to the database. Process 3.4 triggers the Notification Service which emails User B. An alt frame separates the valid path from the invalid path where an error is returned. A loop frame covers the retry scenario. User B accepts, 3.5 updates the database, and a confirmation is sent to User A. Both users can then message and schedule.



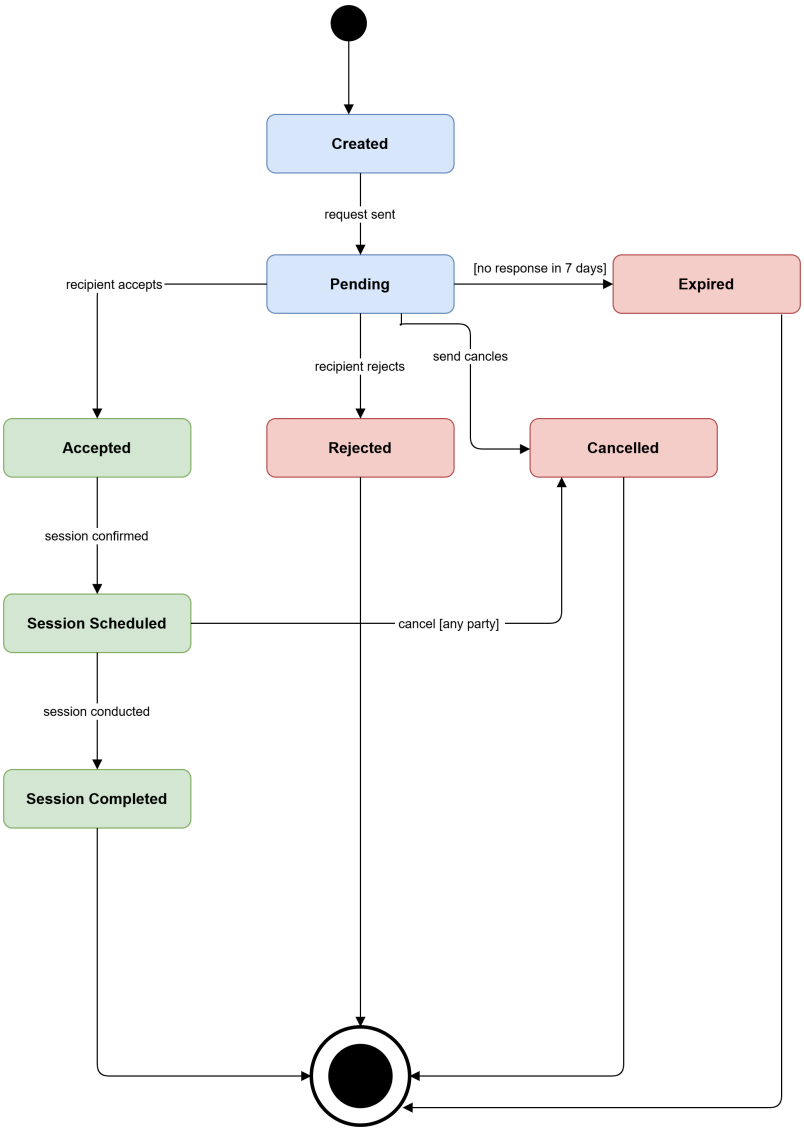
Activity Diagram

The diagram covers the full user flow from login to session completion. After login, a decision splits new users to Complete Profile and existing users straight to Search Users by Skill. Both paths merge at a merge node before search. The user views a profile and sends a request. If rejected, the user retries from the merge node. If accepted, the user messages, schedules, and conducts the session then rates the other party. A dispute check follows. No dispute means the flow ends. A dispute raised sends the user to report, Admin resolves it, and the flow ends.


Register or Login
New or Existing User?
[New User]
Complete Profile
(skills offered and wanted)
[Existing User]
retry
Search Users by Skill
View User Profile
Send Barter Request
Request Accepted?
[No]
Try Another User
[Yes]
Message Matched User
Schedule Session
Conduct Session
Rate and Review
Dispute?
[Dispute]
Report User
Admin Resolves Dispute
[No Dispute]

State Machine Diagram

The diagram models the lifecycle of a Barter Request. It starts in Created, moves to Pending when sent, then branches four ways. Accepting moves it to Accepted, then Session Scheduled, then Session Completed, which is the only path to the final state. Rejecting moves it to Rejected. Cancelling moves it to Cancelled. No response in seven days moves it to Expired. Rejected, Cancelled, and Expired all connect to the final state. A scheduled session can also be cancelled by either party.



Milestone 3: Design and Architecture

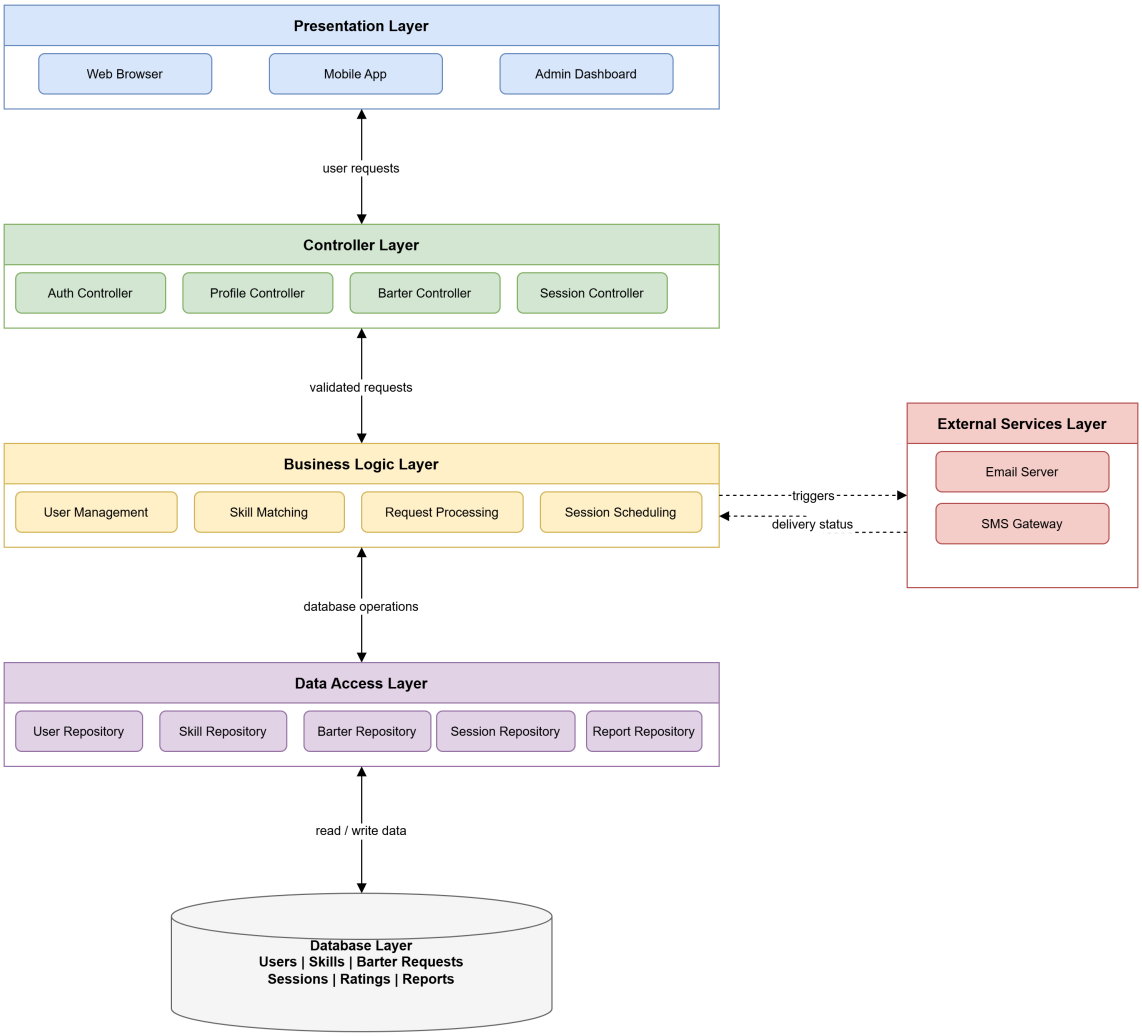
Architecture

We used an N-Tier Layered Architecture. Each layer talks only to the one next to it. Microservices was too heavy for this scale. Event-driven was unnecessary since notifications are direct service calls. MVC principles are applied inside the Controller Layer only.

Layer Breakdown

Layer	Components	Responsibility
Presentation	Web Browser, Mobile App, Admin Dashboard	Renders the interface and collects user input.
Controller	Auth, Profile, Barter, Session, Admin, Notification Controllers	Validates input, checks permissions, routes to the correct service.
Business Logic	User Management, Skill Matching, Request Processing, Session Scheduling, Rating and Review, Dispute Resolution	Runs all core workflows and applies business rules.
Data Access	User, Skill, Barter, Session, Rating, Report Repositories	Wraps all database queries so business logic never writes raw queries.
Database	Users, Skills, Barter Requests, Sessions, Ratings, Reports	Stores all platform data. Only accessible through the Data Access Layer.
External Services	Email Server, SMS Gateway, Notification Service	Sends outbound notifications. Runs independently so a delivery failure does not affect core operations.

Architecture Diagram



Request Flow

A barter request goes from the browser to the Barter Controller, which validates it and passes it to Business Logic. Request Processing checks eligibility, creates the request as Pending, and the Data Access Layer writes it to the database. Business Logic triggers the notification. The user sees a confirmation.

Design Decisions

Decision	Reason
Repository Pattern	Business logic does not touch the database directly. Swapping the database engine only requires changes in the Data Access Layer.
Controller as Gatekeeper	Auth checks and input validation happen in one place. Invalid requests are blocked before reaching business logic.
External Services Isolated	A failed email does not crash a barter request. Core operations complete first, notification is separate.
Strict Layer Order	No layer skips another. Dependencies stay predictable and testable.
Role-Based Access Control	Admin and User roles are enforced at the Controller Layer. A regular user cannot reach admin functions.

Milestone 4: Testing and Management

Test Plan

We tested the platform across three levels. Unit tests target individual functions in isolation. Integration tests verify that two or more components work together correctly. System tests validate complete end-to-end flows against the functional requirements.

Unit Tests

ID	Component	Test Case	Expected Result	Status
UT-01	User Registration	Register with valid name, email, and password	Account created, user redirected to dashboard	Pass
UT-02	User Registration	Register with an already existing email	System returns duplicate email error	Pass
UT-03	User Login	Login with correct credentials	User authenticated and session started	Pass
UT-04	User Login	Login with wrong password	Error shown, account locked after 3 attempts	Pass
UT-05	Profile Management	Update skills offered and skills wanted	Profile saved and visible to other users	Pass
UT-06	Profile Management	Save profile with empty required fields	System blocks save and shows validation error	Pass
UT-07	Skill Search	Search for an existing skill	Matching user profiles returned	Pass
UT-08	Skill Search	Search for a skill with no matches	No results message displayed	Pass
UT-09	Barter Request	Send request with skill selected	Request created with Pending status	Pass
UT-10	Barter Request	Send request without listing a skill	System prompts user to update profile first	Pass
UT-11	Rating	Submit 5-star rating after completed session	Rating saved and displayed on profile	Pass
UT-12	Rating	Skip written review and submit only stars	Stars saved, review field remains empty	Pass

Integration Tests

ID	Components	Test Case	Expected Result	Status
IT-01	Auth + User DB	Register new user and verify record in database	User record stored with hashed password	Pass
IT-02	Barter Request + Notification	Send barter request and verify recipient receives notification	Notification delivered to recipient	Pass
IT-03	Barter Request + Barter DB	Accept request and verify status update in database	Status changed to Accepted in DB	Pass
IT-04	Session + Notification	Schedule session and verify reminder notification sent	Reminder delivered to both users	Pass
IT-05	Rating + Review DB	Submit rating after session and verify record stored	Rating record written to Review Database	Pass
IT-06	Report + Admin Panel	User submits report and verify it appears in admin queue	Report visible in admin dispute queue	Pass
IT-07	Auth + Controller	Unauthenticated user tries to access barter feature	Request blocked at Controller Layer	Pass
IT-08	Admin + User DB	Admin suspends user and verify account status updated	User account marked suspended in database	Pass

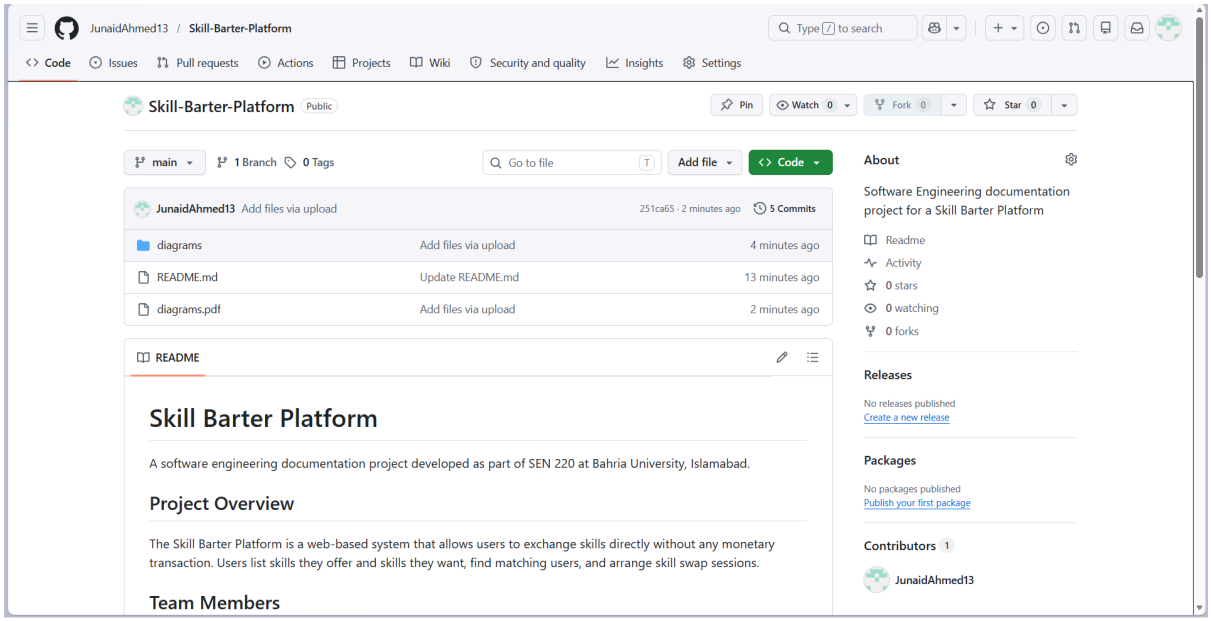
System Tests

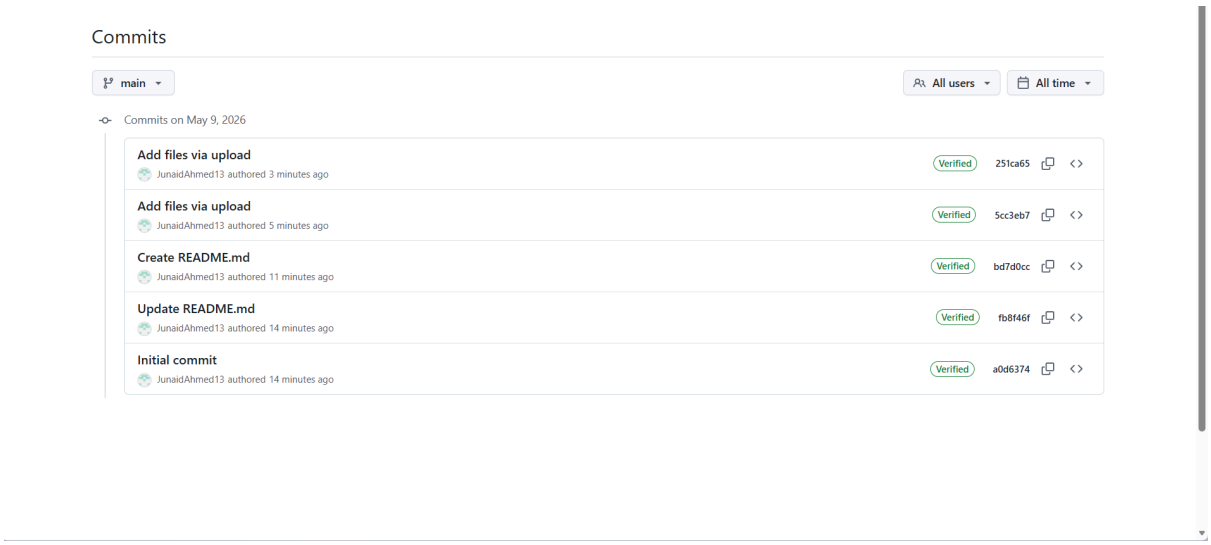
ID	Flow	Test Case	Expected Result	Status
ST-01	Full Barter Flow	User A registers, lists skill, finds User B, sends request, User B accepts, both schedule and complete session	Full flow completes without errors	Pass

ID	Flow	Test Case	Expected Result	Status
ST-02	Dispute Flow	User A files a report after session, Admin reviews and resolves with a warning	Report closed with resolution recorded	Pass
ST-03	Expired Request	User A sends request, User B does not respond for 7 days	Request status changes to Expired automatically	Pass
ST-04	Cancel Flow	User A cancels a pending request before recipient responds	Request status changes to Cancelled	Pass
ST-05	Admin Management	Admin views all active sessions and suspends a flagged user	User suspended and sessions updated	Pass
ST-06	Rating Flow	Both users rate each other after a completed session	Ratings saved and both profile scores updated	Pass

Version Control

We used GitHub for version control throughout the project. All documentation files, diagrams, and LaTeX source files were committed across multiple commits. The repository is available at `https://github.com/JunaidAhmed13/skill-barter-platform`

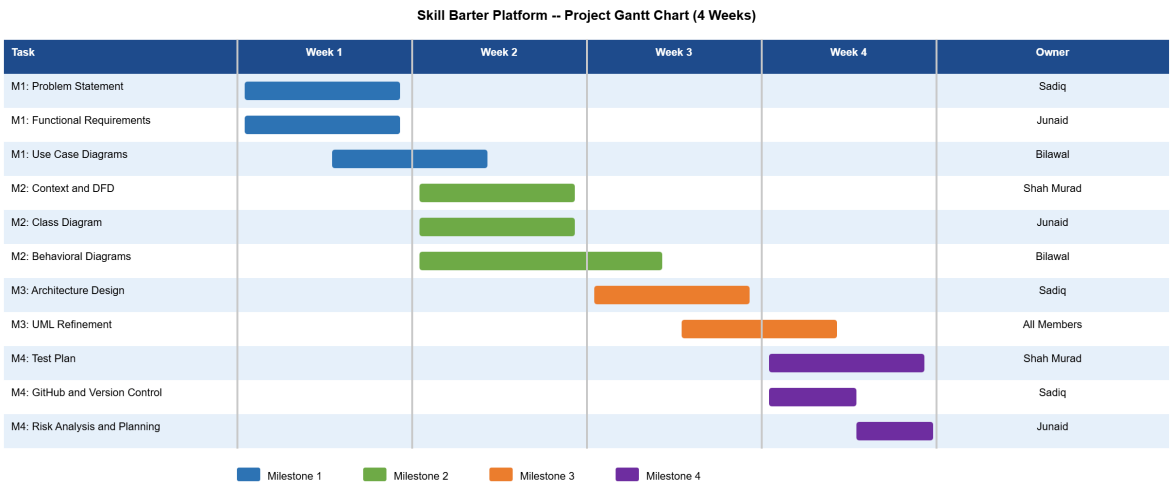




Project Planning

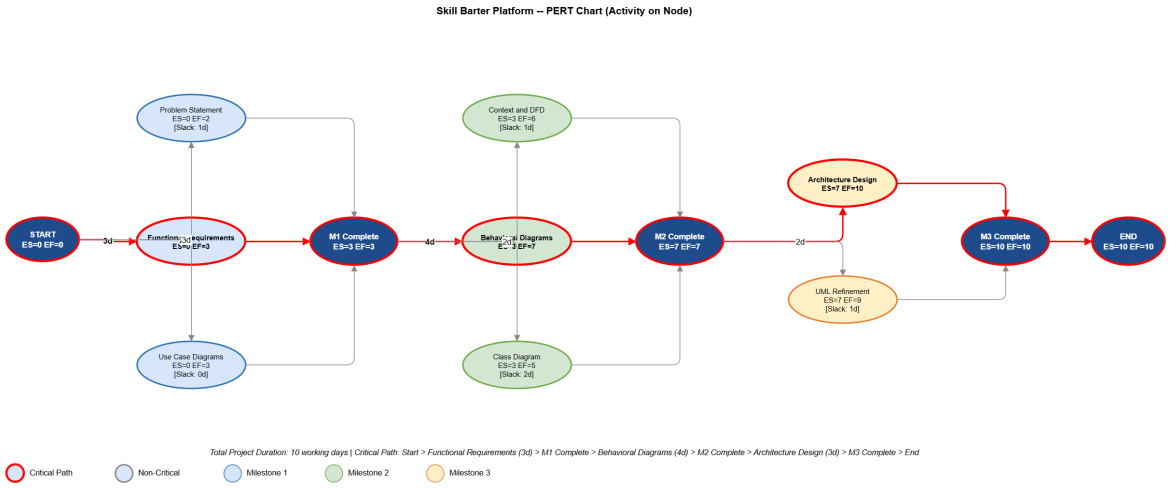
Gantt Chart

Milestone 1 tasks ran through Week 1. Milestone 2 modeling work ran through Weeks 2 and 3. Milestone 3 architecture work ran through Week 3. Milestone 4 testing and documentation was completed in Week 4.



PERT Chart

Total project duration is 10 working days. The critical path runs through Functional Requirements (3 days), Behavioral Diagrams (4 days), and Architecture Design (3 days). Non-critical tasks including Class Diagram, UML Refinement, and Context and DFD carry 1 to 2 days of slack each.



Risk Analysis

ID	Risk	Likelihood	Impact	Priority	Mitigation
R-01	Incomplete requirements	Medium	High	High	Review requirements with all team members before moving to modeling
R-02	Diagram errors or inconsistencies	High	Medium	High	Peer review all diagrams before submission
R-03	Uneven workload distribution	Medium	Medium	Medium	Assign tasks at the start of each milestone with clear deadlines
R-04	Version conflicts in documentation	Low	Medium	Medium	Use GitHub branches and commit frequently
R-05	Scope creep in documentation	Medium	Low	Low	Stick to the assignment brief
R-06	Tool unavailability	Low	High	Medium	Keep local backups of all files and diagrams
R-07	Misunderstanding of SE notation	Medium	High	High	Reference Sommerville and IEEE standards before drawing diagrams
R-08	Late submission due to review delays	Low	High	High	Complete each milestone two days before the deadline

Team Roles

Muhammad Sadiq Aslam Khan (01-134242-091) – Milestone 2: Context Diagram, DFD Level 1 and Level 2, Class Diagram, Sequence Diagram, architecture design, GitHub repository management.

Malik Junaid Ahmed (01-134242-061) – Milestone 1: Functional and non-functional requirements, use case description tables. Milestone 4: Test plan, Gantt chart, PERT chart, risk analysis, team roles.

Bilawal Khokhar (01-134242-036) – Milestone 3: UML refinement, state machine diagram, layered architecture documentation.

Shah Murad (01-134242-113) – Milestone 1: Problem statement, project scope, use case diagrams. Milestone 2: Activity diagram, state machine diagram.